

Práctica 1.2. Sistema de Ficheros

Objetivos

En esta práctica se revisan las funciones del sistema básicas para manejar un sistema de ficheros, referentes a la creación de ficheros y directorios, duplicación de descriptores, obtención de información de ficheros o el uso de cerrojos.

Contenidos

- Preparación del entorno para la práctica
- Creación y atributos de ficheros
- Redirecciones y duplicación de descriptores
- Cerrojos de ficheros
- Directorios

Preparación del entorno para la práctica

La realización de esta práctica únicamente requiere del entorno de desarrollo (compilador, editores y utilidades de depuración). Estas herramientas están disponibles en las máquinas virtuales de la asignatura y en la máquina física de los puestos del laboratorio.

Nota: Si estás usando docker, es posible que algunos de los ejercicios no funcionen correctamente.

Nota 2: Si estás usando WSL, asegúrate de estar trabajando sobre una partición ext4, y no NTFS. Aún así, no se garantiza que todos los ejercicios funcionen correctamente.

Creación y atributos de ficheros

El inodo de un fichero guarda diferentes atributos de éste, como por ejemplo el propietario, permisos de acceso, tamaño o los tiempos de acceso, modificación y creación. En esta sección veremos las llamadas al sistema más importantes para consultar y fijar estos atributos así como las herramientas del sistema para su gestión.

Ejercicio 1. `ls(1)` muestra el contenido de directorios y los atributos básicos de los ficheros. Consultar la página de manual y estudiar el uso de las opciones `-a -l -d -h -i -R -1 -F` y `--color`. Estudiar el significado de la salida en cada caso.

Ejercicio 2. El *modo* de un fichero es `<tipo><rw_usuario><rw_grupo><rw_resto>`:

- tipo: - fichero ordinario; d directorio; l enlace; c dispositivo carácter; b dispositivo bloque; p FIFO; s socket
- rw: r lectura (4); w escritura (2); x ejecución (1)

Comprobar los permisos de algunos directorios (con `ls -ld`).

Ejercicio 3. Los permisos se pueden otorgar de forma selectiva usando la notación octal o la simbólica. Ejemplo, probar las siguientes órdenes (equivalentes):

- `chmod 540 fichero`
- `chmod u=rx,g=r,o= fichero`

¿Cómo se podrían fijar los permisos `rw-r--r-x`, de las dos formas? Consultar la página de manual `chmod(1)` para ver otras formas de fijar los permisos (p.ej. los operadores `+` y `-`).

Ejercicio 4. Crear un directorio y quitar los permisos de ejecución para usuario, grupo y otros. Intentar cambiar al directorio.

Ejercicio 5. Escribir un programa que, usando `open(2)`, cree un fichero con los permisos `rw-r--r-x`. Comprobar el resultado y las características del fichero con `ls(1)`.

Ejercicio 6. Cuando se crea un fichero, los permisos por defecto se derivan de la máscara de usuario (*umask*). Usando el comando interno de la *shell* `umask`, fijar la máscara de forma que los nuevos ficheros no tengan permiso de escritura para el grupo y no tengan ningún permiso para otros. Comprobar el funcionamiento con `touch(1)`, `mkdir(1)` y `ls(1)`.

Ejercicio 7. Modificar el ejercicio 5 para que, antes de crear el fichero, se fije la máscara igual que en el ejercicio 6. Comprobar el resultado con `ls(1)`. Comprobar que la máscara del proceso padre (la *shell*) no cambia.

Ejercicio 8. `ls(1)` puede mostrar el inodo con la opción `-i`. El resto de información del inodo puede obtenerse usando `stat(1)`. Consultar las opciones del comando y comprobar su funcionamiento.

Ejercicio 9. Escribir un programa que emule el comportamiento de `stat(1)` y muestre:

- El número *major* y *minor* asociado al dispositivo.
- El número de inodo del fichero.
- El tipo de fichero (directorio, enlace simbólico o fichero ordinario).
- La hora en la que se accedió el fichero por última vez. ¿Qué diferencia hay entre `st_mtime` y `st_ctime`?

Ejercicio 10. Los enlaces se crean con `ln(1)`:

- Con la opción `-s`, se crea un enlace simbólico. Crear un enlace simbólico a un fichero ordinario y otro a un directorio. Comprobar el resultado con `ls -l` y `ls -li`. Determinar el inodo de cada fichero.
- Repetir el apartado anterior con enlaces rígidos. Determinar con `stat(1)` los inodos de los ficheros y las propiedades, en especial, el número de enlaces.
- ¿Qué sucede cuando se borra uno de los enlaces rígidos? ¿Qué sucede si se borra uno de los enlaces simbólicos? ¿Y si se borra el fichero original?

Ejercicio 11. `link(2)` y `symlink(2)` crean enlaces rígidos y simbólicos, respectivamente. Escribir un programa que reciba una ruta a un fichero como argumento. Si la ruta es un fichero regular, creará un enlace simbólico y rígido con el mismo nombre terminado en `.sym` y `.hard`, respectivamente. Comprobar el resultado con `ls(1)`.

Redirecciones y duplicación de descriptores

La *shell* proporciona operadores (`>`, `>&`, `>>`) que permiten redirigir un fichero a otro (ver los ejercicios propuestos en la práctica opcional). Esta funcionalidad se implementa mediante `dup(2)` y `dup2(2)`.

Ejercicio 12. Escribir un programa que redirija la salida estándar a un fichero cuya ruta se pasa como primer argumento. Probar haciendo que el programa escriba varias cadenas en la salida estándar.

Ejercicio 13. Modificar el programa anterior para que también redirija la salida estándar de error al fichero. Comprobar el funcionamiento incluyendo varias sentencias que impriman en ambos flujos. ¿Hay diferencia si las redirecciones se hacen en diferente orden?

¿Por qué `ls > dirList 2>&1` es diferente a `ls 2>&1 > dirList`? Prueba a cambiar el comando `ls` por `cat <ficheroNoExistente` en el ejemplo anterior.

Cerros de ficheros

Se trata de cerros de ficheros consultivos según el estándar POSIX.

Ejercicio 14. El estado y los cerros de fichero en uso en el sistema se pueden consultar en el fichero `/proc/locks`. Estudiar el contenido de este fichero.

Ejercicio 15. Escribir un programa que intente bloquear un fichero usando `lockf(3)`:

- Si lo consigue, mostrará la hora actual y suspenderá su ejecución durante 10 segundos con `sleep(3)`. A continuación, desbloqueará el fichero, suspenderá su ejecución durante otros 10 segundos y terminará.
- Si no lo consigue, el programa mostrará el error con `perror(3)` y terminará.

Ejecuta el código y trata de escribir en el fichero bloqueado usando el comando `echo`. ¿Es posible? Ahora ejecuta el mismo ejecutable dos veces (de forma simultánea) en diferentes terminales. ¿Qué ocurre en la segunda ejecución?

Ejercicio 16 (Opcional). `flock(1)` proporciona funcionalidad de cerros antiguos BSD en guiones *shell*. Consultar la página de manual y el funcionamiento del comando.

Directorios

Ejercicio 17. Escribir un programa que muestre el contenido de un directorio cuya ruta se proporciona como argumento. Para ello, el programa:

- Cambiará al directorio y comprobará que tiene acceso.
- Recorrerá las entradas del directorio y escribirá su nombre de fichero. Además:
 - Si es un fichero regular y tiene permiso de ejecución para usuario, grupo u otros, escribirá el carácter `*` después del nombre.
 - Si es un directorio, escribirá el carácter `/` después del nombre.
 - Si es un enlace simbólico, escribirá `->` y el nombre del fichero enlazado obtenido con `readlink(2)`.
- Al final, escribirá el tamaño total que ocupan los ficheros regulares en kilobytes.